

Scout Tech Stack, Architecture & 3-Week Plan

Team reference doc — everything in one place: what we're using, why, and who does what.

0. Quick glossary

- **Frontend** = what the user sees and clicks (the website itself)
 - **Backend** = the "behind the scenes" code that handles logic and talks to the database
 - **Database** = where all the data actually lives, permanently
 - **DDL** = *creating* the structure of a table (the empty shelf, before anything is on it)
 - **DML** = *putting data in and taking data out* of a table that already exists (using the shelf)
 - **ORM** = a tool that lets you write Python instead of SQL to talk to the database
 - **Raw SQL** = writing actual SQL by hand, instead of using the ORM shortcut
 - **Auth token / session** = a secure "who is this user" ID that Django checks on every request.
-

1. The Tech Stack

Layer	What we're using	Why, simply
Frontend	React + Tailwind + reactbits.dev components	Looks good fast, AI can help write it — but the team needs a few days to learn basic JavaScript first
Backend	Django (Python)	We already know Python. Django gives us login/auth and an admin dashboard for free — we don't have to build those from scratch

Layer	What we're using	Why, simply
Database	PostgreSQL, hosted on Supabase	Free, hosted, and fits our data — everything is connected (student → documents → applications → opportunities)
How data moves	Django ORM for most things, raw parameterized SQL for 2 specific queries	See section 3
Secrets	<code>.env</code> file	Keeps passwords and API keys out of GitHub so nobody can steal them

2. DDL vs. DML

Step	What it means	Who/what does it	Why
DDL — building the tables	Happens once, at the start, per table. Turning the ERD into real <code>CREATE TABLE</code> statements	We write it by hand , in Supabase's SQL editor	So we can actually see and understand our own database structure, instead of Django auto-generating it as a black box we don't understand
DML — using the tables day-to-day	Happens constantly, for as long as the app exists. Adding a row, reading rows, updating a row	Django's ORM does most of it (<code>Student.objects.filter(...)</code>), except for 2 specific queries (see below)	The ORM saves huge amounts of time on repetitive stuff. We only hand-write SQL for the couple of queries genuinely worth practicing by hand

One extra step this requires — don't skip it. Because we're building tables by hand instead of letting Django build them, every Django model needs one extra

line telling Django "don't touch this table, I've got it":

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()

    class Meta:
        managed = False # tells Django: don't try to auto-
                           create or auto-change this table
```

Without this line, Django might try to "fix" a hand-built table to match what it expects — which can cause errors or overwrite your structure. **Every model needs it.**

3. Which 2 queries get hand-written SQL?

Everything else uses the ORM. Only these two are written as real, hand-typed SQL — because they're the genuinely interesting, relationally complex ones worth practicing, and the kind that ties into our SQL coursework:

1. **Matching students to opportunities** — comparing a student's profile against opportunity requirements
2. **The tracking dashboard** — pulling together a student's applications and their statuses

Everything else — registering a student, updating a profile, uploading a document, bookmarking something — uses the ORM. Less typing, less room for mistakes.

Non-negotiable safety rule: always use `%s` placeholders for data going into a raw query, never paste data straight into the SQL text (this is how SQL injection attacks happen):

```
# Do this:
cursor.execute("SELECT * FROM opportunities WHERE field = %s", [field_of_study])

# Never this:
```

```
cursor.execute(f"SELECT * FROM opportunities WHERE field = '{field_of_study}'")
```

4. How the three layers talk to each other

- **React** (in the browser) → sends a request to → **Django** (the backend)
- **Django** → reads/writes data using the ORM or the hand-written SQL → **PostgreSQL** (the database)
- React **never** talks to the database directly. Only Django does.

Walkthrough: logging in and viewing eligible opportunities

1. Student types email/password into the React form and hits submit. Axios sends it to Django.
2. Django checks the credentials against the `students` table in Supabase (via Psycopg, the driver that lets Python talk to Postgres).
3. If correct, Django creates an **auth session/token** for that student and sends it back to React. React stores *only this token* (not the student's field of study, year, or grades) and redirects to the dashboard.
4. Student clicks "View Eligible Jobs." React sends a request to Django, attaching the auth token — it does **not** send field of study, year, or average. It doesn't need to; it doesn't hold that data.
5. Django reads the token, confirms who the student is, and **looks up that student's current field of study, year, and average directly from the database** — not from anything the browser sent.
6. Django runs the matching raw SQL query using those freshly-looked-up values, and sends the results back to React.

Why this matters (and differs from an earlier draft of this flow): an earlier version of this plan had React store the student's field/year/average in the browser and send those values back to Django as plain URL parameters for the matching query. That's a security gap — anything sent from the browser can be edited by the user, meaning a student could tamper with those values and see opportunities they're not actually eligible for. Having Django look the student up itself (via the auth token) instead of trusting values from the browser

closes that gap, and also means the matching query always uses up-to-date profile data — no risk of the dashboard using stale grades after a profile edit.

5. Non-negotiables (apply to everyone, every week)

- `.env` file for all credentials — never hardcode passwords or keys into `settings.py`, never commit `.env` to GitHub
 - Every raw SQL query uses `%s` placeholders — never Python f-strings
 - Sensitive fields (ID numbers, banking details, health info) stay structurally separate from professional profile data (POPIA/FR8/NFR1)
 - Every hand-built table's Django model has `managed = False`
 - Matching/eligibility data is always looked up server-side by Django from the database — never trusted from the browser
-

6. Three-Week Plan

What's due at the checkpoint: a progress report + partial code + a created database. The full app is due in October — the goal these 3 weeks is *real, visible progress*, not a finished product.

Team split (6 people): 3 on frontend, 3 on backend/database — working in parallel, not sequentially, so nobody sits idle waiting on someone else.

Week 1 — Foundations (parallel tracks)

Frontend team:

- Small JS/React crash course (state, props, one `fetch` /Axios call) before building real screens
- Set up React + Vite + Tailwind project, pull in reactbits.dev components for layout
- Build the registration and profile screens using **mock/fake data** (not connected to anything real yet)

Backend/DB team:

- Turn the ERD into real tables: hand-write `CREATE TABLE` statements in Supabase's SQL editor
- Set up the Django project, connect it to Supabase via `.env` credentials

- Build Django models matching the hand-created tables (`managed = False` on each)
- Get one working end-to-end endpoint (e.g. `GET /opportunities`) to prove the connection works

Note:

Django models are the *structure* — a Python class describing what a Student or Opportunity looks like (its fields/columns). This mirrors your hand-written database tables. It's a blueprint, not an action.

End of Week 1 checkpoint: database tables exist in Supabase, Django connects to them successfully, React project runs with basic (mock-data) screens.

Week 2 — Core Features

Backend team:

- Build registration/auth endpoints (ORM-based), using Django's built-in auth for sessions/tokens
- Build profile management endpoints (ORM-based)
- Build the eligibility-matching endpoint using raw parameterized SQL — Django looks up the student's data itself via their auth token, per section 4 - must have
- Test every endpoint in Postman before frontend integration begins

Frontend team:

- Swap the registration/profile screens from mock data to real Django endpoints
- Build the opportunity listing/filter screen, connected to the real matching endpoint
- Handle basic error states (failed login, missing fields)

End of Week 2 checkpoint: a student can realistically register, build a profile, and see a list of matched opportunities — even if the UI isn't fully polished.

Week 3 — Integration, Polish & Submission Prep

Whole team:

- Build the application-tracking dashboard endpoint (second raw SQL query) and its frontend screen
- Full integration pass — make sure frontend and backend are correctly talking end-to-end; fix bugs found
- Write the progress report: what's built, what's in progress, architecture decisions and why (this doc is a strong starting point), screenshots of the working database schema
- Package partial code for submission: clean commits, confirm `.env` is not committed, README with setup instructions

Due at 3 weeks: progress report + partial working code + a created, populated database.

7. Deliberately deferred past the 3-week checkpoint

To keep the 3 weeks realistic, these carry on through to October rather than needing to be done by the checkpoint:

- Auto-apply / assisted application flow (FR3, FR4)
- Receipts & downloadable summary reports (FR5)
- Bookmarking & deadline reminders (FR7)
- Admin/provider management screens (FR10) — Django's built-in admin panel can serve as a placeholder in the meantime
- Full responsive design polish (NFR3)